Файлы CSV

Теория

```
date;film;price;sold
01.10;Лето;350,5;42
02.10;"Он; она";280;9
03.10;Свет;300;31
```



Всё ниже — на двух учебных файлах. Примеры идут подряд, как ячейки ноутбука: что объявлено выше, тем пользуются ниже. Под кодом, с чертой слева, — ровно то, что он печатает.

Формат

CSV — текст: строка файла — строка таблицы, поля через разделитель, первая строка обычно заголовок. Поле с разделителем, кавычкой или переводом строки берут в двойные кавычки, кавычку внутри удваивают. Файл `films.csv`:

```
id,title,genre,year,rating
1,Северный ветер,драма,1998,7.4
2,"Город, где светло",комедия,2004,6.9
3,Лето, Драма ,2011,8.0
4,Мост,триллер,2004,7.1
5,Остров,драма,2006,
6,Свет,комедия,2015,н/д
```

Поэтому строку не режут по запятой руками — кусков выходит больше, чем полей:

```
line = '2,"Город, где светло",комедия,2004,6.9'
parts = line.split(",")
print(len(parts), parts[:3])
```

```
6 ['2', '"Город', ' где светло"']
```

pandas кавычки понимает:

```
import pandas as pd
films = pd.read_csv("films.csv")
print(films)
print(films.dtypes)
```

```
   id              title    genre  year rating
0   1     Северный ветер    драма  1998    7.4
```

```
1   2  Город, где светло  комедия  2004    6.9
2   3               Лето   Драма   2011    8.0
3   4               Мост  триллер  2004    7.1
4   5             Остров    драма  2006    NaN
5   6               Свет  комедия  2015    н/д
id        int64
title       str
genre       str
year      int64
rating      str
dtype: object
```

Пустое поле стало пропуском `NaN`, а `н/д` — нет, и весь столбец `rating` получил тип `str`: числа в нём не посчитать. В pandas 2 строковый тип печатается как `object`.

Пропуски

Что ещё считать пропуском, говорят при чтении:

```
films = pd.read_csv("films.csv", na_values=["н/д"])
r = films["rating"]
print(r.dtype, r.isna().sum())
```

```
float64 2
```

Или превращают в числа потом: всё, что не число, станет `NaN`.

```
raw = pd.read_csv("films.csv")
r = pd.to_numeric(raw["rating"], errors="coerce")
print(r.tolist())
```

```
[7.4, 6.9, 8.0, 7.1, nan, nan]
```

`mean`, `sum`, `median` пропуски пропускают: среднее считается по четырём оценкам, а не по шести.

Русский Excel

Excel с русскими настройками пишет CSV по-своему: разделитель — точка с запятой, десятичная — запятая, кодировка — UTF-8 с меткой BOM в начале файла или cp1251. Файл `bilety.csv`, продажи билетов на сеансы:

```
date;hall;film_id;price;sold
01.10.2026;1;3;350,5;42
01.10.2026;2;1;300;18
02.10.2026;1;3;350,5;57
02.10.2026;2;4;280;9
03.10.2026;1;2;300;31
03.10.2026;2;7;250;12
```

Сначала файл смотрят как текст. `repr` показывает разделитель, кавычки и невидимое:

```
with open("bilety.csv", encoding="utf-8") as f:
    print(repr(f.read(40)))
```

```
'\ufeffdate;hall;film_id;price;sold\n01.10.2026'
```

`\ufeff` в начале — это BOM. Прочитанный как попало, файл склеивается в один столбец — выдаёт размер:

```
df = pd.read_csv("bilety.csv")
print(df.shape)
print(df.columns.tolist())
```

```
(6, 1)
['date;hall;film_id;price;sold']
```

Правильно:

```
df = pd.read_csv("bilety.csv", sep=";", decimal=",",
                 encoding="utf-8-sig")
print(df.head(3))
print(df.dtypes)
```

```
         date  hall  film_id  price  sold
0  01.10.2026     1        3  350.5    42
1  01.10.2026     2        1  300.0    18
2  02.10.2026     1        3  350.5    57
date           str
hall         int64
film_id      int64
price      float64
sold         int64
dtype: object
```

`UnicodeDecodeError` при чтении — не та кодировка: пробовать `encoding="cp1251"`.

Параметры чтения

- `sep=";"`, `decimal=","` — разделитель и десятичная;
- `encoding="utf-8-sig"` — съедает BOM, `"cp1251"` — старый русский Excel;
- `na_values=["-", "н/д"]` — что ещё считать пропуском;
- `dtype={"code": str}` — не терять ведущие нули;
- `header=None, names=[...]` — файл без заголовка;
- `usecols=[...]` — только нужные столбцы;
- `skipinitialspace=True` — пробелы после разделителя.

Проверить чтение на куске текста, не заводя файла, — `StringIO`:

```
from io import StringIO
text = "code,n\n007,1\n042,2\n"
print(pd.read_csv(StringIO(text))["code"].tolist())
print(pd.read_csv(StringIO(text),
                  dtype={"code": str})["code"].tolist())
```

```
[7, 42]
['007', '042']
```

Первый осмотр

Сразу после чтения:

```
print(films.shape)
print(films.isna().sum())
print(films["year"].describe())
```

```
(6, 5)
id        0
title     0
genre     0
year      0
rating    2
dtype: int64
count       6.000000
mean     2006.333333
std         5.955390
min      1998.000000
25%      2004.000000
50%      2005.000000
75%      2009.750000
max      2015.000000
Name: year, dtype: float64
```

- `df.head()`, `df.dtypes`, `df.columns.tolist()` — что за столбцы и какого типа.
- Столбец чисел с типом `str` или `object` — в нём есть не число.
- Имя столбца с пробелом или BOM, `" price"`, `df["price"]` не найдёт: `KeyError`.
- `s.value_counts()` — какие значения и сколько раз, `s.nunique()` — сколько различных.

Чистка строк

Значение с пробелом или другой буквой — отдельное значение:

```
print(films["genre"].value_counts())
films["genre"] = films["genre"].str.strip().str.lower()
print(films["genre"].value_counts())
```

```
genre
драма      2
комедия    2
 Драма     1
триллер    1
Name: count, dtype: int64
genre
драма      3
комедия    2
триллер    1
Name: count, dtype: int64
```

Ещё: `s.str.replace(a, b)`, `s.str.contains("ет")`, `s.str.len()`; повторы строк — `df.drop_duplicates()`.

Выборка и сортировка

Каждое условие — в скобках, «и» — `&`, «или» — `|`, «не» — `~`:

```
d = films[(films["genre"] == "драма")
          & (films["year"] >= 2000)]
print(d[["title", "year"]])
kom = films[films["genre"].isin(["комедия", "триллер"])]
print(kom["title"].tolist())
```

```
    title  year
2    Лето  2011
4  Остров  2006
['Город, где светло', 'Мост', 'Свет']
```

```
top = films.sort_values("rating", ascending=False)
print(top[["title", "rating"]].head(3))
```

```
            title  rating
2            Лето     8.0
0  Северный ветер     7.4
3            Мост     7.1
```

Пропуски при сортировке уходят в конец. Несколько ключей — `sort_values(["a", "b"], ascending=[False, True])`. `s.between(a, b)` включает оба конца.

Группы

```
g = films.groupby("genre")["rating"].agg(
    ["mean", "count", "size"])
print(g.round(2))
```

```
          mean  count  size
genre
драма      7.7      2     3
комедия    6.9      1     2
триллер    7.1      1     1
```

`count` не считает пропуски, `size` считает: в драмах три фильма, оценок две. `transform` возвращает значение группы на каждой строке — столбец той же длины, что таблица:

```
gr = films.groupby("genre")["rating"]
films["sred"] = gr.transform("mean")
print(films[["title", "genre", "sred"]].round(2))
```

```
                title    genre  sred
0     Северный ветер    драма   7.7
1  Город, где светло  комедия   6.9
2               Лето    драма   7.7
3               Мост  триллер   7.1
4             Остров    драма   7.7
5               Свет  комедия   6.9
```

Два файла

Соединение по ключу. `how="left"` — все строки левой таблицы; нет пары — пропуск. `indicator=True` показывает, у кого пара нашлась:

```
b = pd.read_csv("bilety.csv", sep=";", decimal=",",
                encoding="utf-8-sig")
m = b.merge(films, left_on="film_id", right_on="id",
            how="left", indicator=True)
print(m[["film_id", "title", "_merge"]])
```

```
   film_id              title     _merge
0        3               Лето       both
1        1     Северный ветер       both
2        3               Лето       both
3        4               Мост       both
4        2  Город, где светло       both
5        7                NaN  left_only
```

```
m["vyruchka"] = m["price"] * m["sold"]
print(m.groupby("title")["vyruchka"].sum())
```

```
title
Город, где светло     9300.0
Лето                 34699.5
Мост                  2520.0
Северный ветер        5400.0
Name: vyruchka, dtype: float64
```

Сеанс фильма 7 в сумму не попал: строки с пропуском в ключе `groupby` выбрасывает, оставить — `dropna=False`. Ключи должны быть одного типа: `42` и `"0042"` не совпадут.

Даты

Формат даты — явно. Без него pandas читает `01.10.2026` как 10 января, а на `13.10.2026` падает с ошибкой.

```
b["date"] = pd.to_datetime(b["date"], format="%d.%m.%Y")
print(b["date"].dt.dayofweek.tolist())
print((b["date"].max() - b["date"].min()).days)
```

```
[3, 3, 4, 4, 5, 5]
2
```

`dayofweek`: 0 — понедельник, 6 — воскресенье. Ещё `.dt.day`, `.dt.month`, `.dt.hour`. Разность дат — `Timedelta`, дни в ней — `.days`.

Ответ

Округление — своими руками. Средняя оценка — ровно 7,35, школьное до десятых — 7,4. Встроенные округления дают разное:

```
s = films["rating"].mean()
print(s, round(s, 1), round(float(s), 1))
print(pd.Series([6.65]).round(1).tolist())
from decimal import Decimal, ROUND_HALF_UP
print(Decimal(str(s)).quantize(Decimal("0.1"),
                               rounding=ROUND_HALF_UP))
```

```
7.35 7.4 7.3
[6.6]
7.4
```

`round` от обычного `float` смотрит на двоичную запись: 7,35 в ней чуть меньше 7,35, и выходит 7,3. numpy и pandas округляют половину к чётному: 6,65 → 6,6. Школьное — только через `Decimal` от строки.

Ответ файлом — без индекса:

```
res = m.groupby("title", as_index=False)["sold"].sum()
res.to_csv("answer.csv", index=False)
print(open("answer.csv", encoding="utf-8").read())
```

```
title,sold
"Город, где светло",31
Лето,99
Мост,9
Северный ветер,18
```

Поле с запятой pandas сам взял в кавычки. Столбцы и их порядок — как в условии.

Без pandas

Модуль `csv` из стандартной библиотеки понимает кавычки так же:

```
import csv
with open("films.csv", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(len(rows), rows[1]["title"])
with open("bilety.csv", encoding="utf-8-sig") as f:
    head = next(csv.reader(f, delimiter=";"))
print(head)
```

```
6 Город, где светло
['date', 'hall', 'film_id', 'price', 'sold']
```

Всё в строке — строки: число из поля — `float(r["rating"])`.

База Python

Справа после # — что даёт выражение.

Числа

```
7 / 2       # 3.5
7 // 2      # 3, целая часть
-7 // 2     # -4, вниз, а не к нулю
7 % 3       # 1
2 ** 10     # 1024
0.1 + 0.2   # 0.30000000000000004
int("42")   # 42
int(3.9)    # 3, дробь отбрасывается
round(2.5)  # 2: половина идёт к чётному
```

Строки

```
s = "Город, где светло"
s[0]                  # 'Г'
s[-1]                 # 'о', с конца
s[:5]                 # 'Город', символы 0..4
len(s)                # 17
s.split(", ")         # ['Город', 'где светло']
s.lower()             # 'город, где светло'
" a b ".strip()       # 'a b'
";".join(["1", "2"])  # '1;2'
f"{2 / 3:.3f}"        # '0.667'
```

Списки

```
a = [3, 1, 2]
a.append(5)
a                             # [3, 1, 2, 5]
a[1:3]                        # [1, 2]
sorted(a)                     # [1, 2, 3, 5], новый список
sorted(a, reverse=True)       # [5, 3, 2, 1]
max(a), sum(a), len(a)        # (5, 11, 4)
[x * x for x in a if x > 2]   # [9, 25]
```

```
list(enumerate("аб"))        # [(0, 'а'), (1, 'б')]
a.sort()                     # None, сортирует сам a
```

Словари и множества

```
d = {"драма": 3}
d["комедия"] = 2
d.get("триллер", 0)             # 0, ключа нет
"драма" in d                    # True
list(d.items())       # [('драма', 3), ('комедия', 2)]
len(set([1, 1, 2]))             # 2, различных
from collections import Counter
Counter("аббаа").most_common(1)  # [('а', 2)]
```

Циклы, функции, ввод и вывод

```
for i in range(3):    # i = 0, 1, 2
    print(i)

def sred(a):
    return sum(a) / len(a)

n = int(input())
a = list(map(int, input().split()))
print(*a)             # через пробел
print(f"{x:.3f}")     # три знака после точки
```